Basics Day 10 — Session 10 (Hours 37–40)

OOP Fundamentals — Classes · Object Collections · Encapsulation · Checkpoint 5

Session 10 Overview

Topics Covered Today

- Hour 37: Intro to OOP — classes, objects, `self`, and methods
- Hour 38: Collections of objects + searching by attribute
- Hour 39: Basic encapsulation + data validation
- Hour 40: Checkpoint 5 — functions, modules, and intro OOP

Hour 37: Intro to OOP — Classes, Objects, and `self`

Learning Outcomes

- Create a basic class using the `class` keyword and `__init__`
- Instantiate objects from a class and call their methods
- Distinguish a **class** (blueprint) from an **object** (instance)
- Use `self` to store and access instance attributes
- Understand why `self` is required in every instance method

Class vs. Object — The Blueprint Analogy

A Class Is a Blueprint

```
Cookie Cutter (class)  →  Cookie 1, Cookie 2, Cookie 3 (objects/instances)
```

Key Points

- A class **defines** structure and behavior — no object exists yet
- An **object** (instance) is a concrete value built from the class
- Multiple independent objects can be created from one class
- Each object has its own copy of the class's attributes

```python
class Contact:
    pass

alice   = Contact()   # object 1
bob     = Contact()   # object 2
```

Defining a Class — `__init__`

The Constructor Pattern

```
class Contact:
    def __init__(self, name, phone):
        self.name  = name    # instance attribute
        self.phone = phone   # instance attribute
```

Creating Objects

```
alice = Contact("Alice Johnson", "555-0001")
bob   = Contact("Bob Smith",    "555-0002")

print(alice.name)   # Alice Johnson
print(bob.name)     # Bob Smith  (independent copy)
```

Why `__init__`?

Instance Attributes and `self`

What `self` Means

- `self` is a reference to the **current object** the method is running on
- When you call `alice.display()`, Python passes `alice` as `self`
- `self.name` means "this object's own `name` attribute"

```
class Contact:
    def __init__(self, name, phone):
        self.name  = name       # belongs to THIS object
        self.phone = phone      # belongs to THIS object

alice = Contact("Alice", "555-0001")
bob   = Contact("Bob",   "555-0002")

# Python internally does:
# Contact.__init__(alice, "Alice", "555-0001")
# Contact.__init__(bob,   "Bob",   "555-0002")
```

Methods — Defining Behavior

A `BankAccount` Class with Multiple Method Types

```
class BankAccount:
    def __init__(self, account_holder, initial_balance):
        self.account_holder = account_holder
        self.balance = initial_balance

    # Action method — modifies state
    def deposit(self, amount):
        self.balance = self.balance + amount
        print(f"Deposited ${amount}. New balance: ${self.balance}")

    # Action method with a safety check
    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance = self.balance - amount
            print(f"Withdrew ${amount}. New balance: ${self.balance}")
        else:
            print(f"Insufficient funds. Balance is ${self.balance}")

    # Query method — returns state without changing it
```

Live Demo — Contact Class

Full Demo Class

```python
class Contact:
    """A simple contact record."""

    def __init__(self, name, phone):
        self.name  = name
        self.phone = phone

    def display(self):
        print(f"Name:  {self.name}")
        print(f"Phone: {self.phone}")

    def call_info(self):
        return f"Calling {self.name} at {self.phone}"

# Create two objects and use them
alice = Contact("Alice Johnson", "555-0001")
bob   = Contact("Bob Smith",    "555-0002")

alice.display()
print(bob.call_info())

# Verify independence — changing alice does not affect bob
alice.phone = "555-9999"
```

Lab — BankAccount Class

Time: 25–35 minutes

Build Progressively Through 7 Checkpoints

Checkpoint	What to Add
1	`__init__(account_holder, initial_balance)`
2	`display_account()` — print holder and balance
3	`deposit(amount)` — add to balance
4	`withdraw(amount)` — subtract with insufficient-funds guard
5	`get_balance()` — return current balance
6	`transfer_to(recipient, amount)` — move money between two accounts

Common Pitfalls — Hour 37

Pitfall 1: Forgetting `self` in the Method Signature

```
# Wrong — TypeError when called
class Contact:
    def __init__(name, phone):   # self missing!
        self.name = name         # NameError: self is not defined
```

```
# Right
class Contact:
    def __init__(self, name, phone):
        self.name  = name
        self.phone = phone
```

Pitfall 2: Class Attribute vs. Instance Attribute

```
# Wrong — ALL instances share the same list!
class Container:
```

Quick Check — Hour 37

Question: What is the difference between a **class** and an **object**?

Expected answer: A class is a blueprint or template. An object (instance) is a concrete value built from that blueprint. You can create many independent objects from one class — each with its own attribute values.

Hour 38: Collections of Objects + Searching

Learning Outcomes

- Search a list of objects by comparing their attributes
- Update an object's attribute in place — no "save back" needed
- Use `.lower()` to make searches case-insensitive
- Wrap search logic in a reusable function returning the object or `None`

The Core Search Pattern

Loop and Compare Attributes

```
class Contact:
    def __init__(self, name, phone):
        self.name  = name
        self.phone = phone

directory = [
    Contact("Alice",   "555-0101"),
    Contact("Bob",     "555-0202"),
    Contact("Charlie", "555-0303"),
]

query = "bob"
found_contact = None

for person in directory:
    # compare the attribute (string), not the whole object
    if person.name.lower() == query.lower():
        found_contact = person
        break

if found_contact:
    print(f"Found them! Phone: {found_contact.phone}")
```

String Normalization and Updating In Place

Always Normalize Both Sides

```
# Without normalization — misses "BOB", "bob", "BoB"
if person.name == "Bob":

# With normalization — catches all variations
if person.name.lower() == query.lower():
```

Updating the Found Object

Once you hold the reference, assign directly — the list sees the change automatically:

```
if found_contact:
    print(f"Old phone: {found_contact.phone}")
    found_contact.phone = "555-9999"   # updates the SAME object in the list
    print(f"New phone: {found_contact.phone}")

# Verify the list sees the change — no "save back" needed
```

search_contact() — A Reusable Function

Wrap the Loop

```
def search_contact(contact_list, target_name):
    """
    Search for a contact by name (case-insensitive).
    Returns the Contact object if found, or None if not found.
    """
    for c in contact_list:
        if c.name.lower() == target_name.lower():
            return c      # returns the actual object immediately

    return None           # no match found
```

Using the Function

```
result = search_contact(directory, "Alice")

if result:
```

Lab — Refactor Contact Manager to Objects

Time: 30 minutes

Replace Dictionaries With Objects

```
[1] Add        [2] List        [3] Search/Update        [4] Quit
```

Lab Requirements

1. Define `Contact` with `__init__(name, phone)` and a `display()` method returning `"Alice -- 555-0101"`
2. Create `my_directory = []` as the in-memory store
3. Add contacts via user input — create a `Contact` object and `append` it
4. List contacts by calling `.display()` on each
5. Search by name (case-insensitive) — show current phone — optionally update in

Common Pitfalls — Hour 38

Pitfall 1: Comparing the Whole Object Instead of an Attribute

```
# Wrong — compares a string to a Contact object
if query == contact:
    ...

# Right — compare the string to the name attribute
if query.lower() == contact.name.lower():
    ...
```

Pitfall 2: Forgetting `break` After a Match

```
# Wrong — loop continues; may overwrite found_contact
for c in directory:
    if c.name.lower() == target.lower():
        found_contact = c          # found it, but loop keeps going!

# Right — stop immediately on match
```

Quick Check — Hour 38

Question: If you set `found_contact.phone = "555-new"`, does the original object in the list change?

Expected answer: Yes. `found_contact` points to the same object that is inside the list. You are modifying the actual object, not a copy. No "save back" step is needed.

Hour 39: Basic Encapsulation + Data Validation

Learning Outcomes

- Write a simple validation method inside a class
- Distinguish core business logic (class) from UI logic (menu loop)
- Protect data by updating `self.phone` only when validation passes
- Return `True` / `False` from a method to signal success or failure
- Preview `ValueError` as an alternative signal for invalid data

Encapsulation — The Core Idea

Analogy — A Bank Account

The bank doesn't let you set your balance directly. You go through `deposit()` and `withdraw()`, which enforce rules.

The Same Pattern for Our Class

```
# No guardrails — garbage accepted silently
c = Contact("Alice", "555-1234")
c.phone = user_input     # anything goes in, no checks

# Method-guarded update
success = c.update_phone(user_input)   # class decides if it is valid
if success:
    print("Update successful!")
else:
    print("Invalid phone number.")
```

UI vs. Core Logic — Separation of Concerns

Two Separate Jobs

Layer	Responsibility
Class (core logic)	Knows what makes a Contact valid; enforces rules; protects data
Menu loop (UI)	Talks to user; collects input; calls class methods; reacts to True/False

Why This Separation Matters

- **Reusable** — any code that creates contacts uses the same validation
- **Maintainable** — change validation rules in one place only
- **Testable** — test the class without running a menu loop

Boolean Validation Method

Validate First, Then Update

```
class Contact:
    def __init__(self, name, phone):
        self.name  = name
        self.phone = phone

    def update_phone(self, new_phone):
        """
        Update phone if it passes validation.
        Returns True if valid and updated, False otherwise.
        """
        # Clean up separators before counting digits
        clean = new_phone.replace(" ", "").replace("-", "")

        # Rule 1: must contain only digits after cleanup
        if not clean.isdigit():
            return False

        # Rule 2: must be at least 7 digits
        if len(clean) < 7:
            return False

        # All rules passed — safe to update
```

Wiring Validation to the Menu

Clean Menu Loop

```
my_contact = Contact("Alice", "555-1234")

while True:
    print(f"\nCurrent phone: {my_contact.phone}")
    user_input = input("Enter new phone (or 'q' to quit): ")

    if user_input.lower() == 'q':
        break

    success = my_contact.update_phone(user_input)

    if success:
        print("Phone updated successfully!")
    else:
        print("Invalid phone number. Must be at least 7 digits.")
```

ValueError Preview (Advanced Teaser)

Lab — Add Validation to Your Contact Manager

Time: 28 minutes

Task Steps

1. Add `update_phone(new_phone)` to your `Contact` class
2. Implement at least one validation rule (length of at least 7 digits)
3. Return `True` if valid and updated; `False` if invalid — do NOT update `self.phone` on failure
4. Update your menu to call `update_phone()` — not direct attribute assignment
5. Test with valid **and** invalid inputs

Lab Checkpoints

- ✓ `update_phone` exists and takes `new_phone` as a parameter
- ✓ Method contains at least one validation `if` check

Common Pitfalls — Hour 39

Pitfall 1: Validation Logic in the Menu, Not the Class

```
# Wrong — validation scattered in UI; must be duplicated everywhere
phone = input("Enter phone: ")
if len(phone) >= 7:
    contact.phone = phone   # direct assignment bypasses class rules
```

```
# Right — validation inside the class method
phone = input("Enter phone: ")
if contact.update_phone(phone):
    print("Updated!")
else:
    print("Invalid — try again.")
```

Pitfall 2: Forgetting to Return a Boolean

```
# Wrong — method returns None on success (None is falsy)
def update_phone(self, new_phone):
```

Quick Check — Hour 39

Question: Where should validation logic live — inside the class or in the menu loop?

Expected answer: Inside the class. The class owns the data, so it owns the rules about what data is valid. Putting validation in the menu loop means duplicating those rules everywhere the data is created or updated. Centralizing validation in the class means one place to maintain — all callers benefit automatically.

Hour 40: Checkpoint 5 — Functions, Modules, and Intro OOP

Learning Outcomes

- Design and build a menu-driven CLI app that loops until quit
- Organize code across multiple files (`main.py`, `utils.py`)
- Use a custom class to represent a domain record with state and behavior
- Implement add, list, and search features with graceful error handling
- Troubleshoot common import errors

Why Multi-File Architecture?

Restaurant Analogy

File	Role
`main.py`	Server — talks to user, orchestrates actions
`utils.py`	Kitchen — reusable helpers any part of the app can call
`record.py`	Recipe definition — what a record looks like and can do

Import Flow is One-Directional

```
main.py  →  utils.py
main.py  →  record.py
```

`utils.py` and `record.py` do **not** import from `main.py` — this keeps each file self-contained and reusable.

The Record Class Pattern

Checkpoint 5 Reference Implementation

```python
# record.py

class Record:
    def __init__(self, name, note):
        self.name = name.strip()
        self.note = note.strip()

    def matches(self, term):
        """Return True if term appears in name or note (case-insensitive)."""
        normalized = term.strip().lower()
        return normalized in self.name.lower() or normalized in self.note.lower()

    def display(self):
        return f"{self.name} — {self.note}"
```

Why a Class Instead of a Dictionary?

- Structure is **enforced** — every record always has `name` and `note`

Menu Loop Skeleton

Start Here, Then Fill In Each Feature

```python
# main.py

def main():
    records = []

    while True:
        print("\n=== Menu ===")
        print("1) Add")
        print("2) List")
        print("3) Search")
        print("4) Quit")

        choice = input("Choose: ").strip()

        if choice == "1":
            print("Add selected")      # replace with real add logic
        elif choice == "2":
            print("List selected")     # replace with real list logic
        elif choice == "3":
            print("Search selected")   # replace with real search logic
        elif choice == "4":
            print("Goodbye.")
            break
        else:
            print("Invalid choice.")
```

Checkpoint 5 Lab Requirements

Time: 40 minutes

Must-Have Deliverables

Item	Requirement
`main.py`	Entry point — menu loop until quit
`utils.py`	At least 2 helper functions imported into `main.py`
Class	At least 1 class with `__init__` plus 1 other method
Functions	At least 3 named functions outside class methods
Features	Add / List / Search all work correctly
Graceful errors	Invalid menu choice handled; blank name rejected
No file I/O	All records in memory only

Common Pitfalls — Hour 40

Pitfall 1: ImportError from Wrong Working Directory

```
ModuleNotFoundError: No module named 'utils'
```

All files must be in the **same folder**. Run Python from that folder:

```
project/
├── main.py
├── utils.py     # same folder as main.py
└── record.py
```

Pitfall 2: File Name Collides with Standard Library

```
# You created math.py — Python imports YOURS, not the built-in
import math
math.sqrt(9)   # AttributeError: module 'math' has no attribute 'sqrt'
```

Quick Check — Hour 40

Question: Point to one place in your code where using a function reduced duplication.

Example answer: "I used `search_records()` from `utils.py` in both my Update and Delete menu options. Without it, I would have copied the same loop and `.lower()` comparison into two separate `elif` blocks. By extracting it once into a named function, both features share the same logic. If I change how searching works, I only update it in one place — this is the DRY principle: Don't Repeat Yourself."

Scope Guardrails

Stay in Basics Scope

✓ Classes with `__init__` and instance methods
✓ Instance attributes via `self`
✓ Multi-file projects with straightforward imports
✓ Validation methods returning `True` / `False`
✓ In-memory data storage (lists of objects)

Not Yet — Advanced Topics

✗ Inheritance and subclassing
✗ Dunder methods beyond `__init__` (`__str__`, `__repr__`, `__eq__`)
✗ `@property`, `@classmethod`, `@staticmethod` decorators
✗ Abstract base classes (`ABC`)
✗ `@dataclass` shortcut

Session 10 Wrap-Up

What We Covered Today

Hour	Topic	Key Takeaway
37	Intro to classes — objects, attributes, methods	A class is a blueprint; objects are independent instances built from it
38	Collections of objects + searching	Loop, compare attributes, normalize strings, update in place
39	Basic encapsulation + validation	Validation belongs in the class; UI only checks the return value
40	Checkpoint 5 — modules and OOP	One file, one responsibility; import flow is one-directional

What's Next — Day 11

Session 11 (Hours 41–44)

- **Hour 41** — File I/O — reading files with `open()` and `with`
- **Hour 42** — File I/O — writing and appending files
- **Hour 43** — Working with CSV files using `csv.reader` and `csv.writer`
- **Hour 44** — Persistent CLI App — combine file I/O with your Day 10 class

Homework / Practice

1. Add `update_phone` validation to today's `Contact` class if not already done
2. Extend `search_contact()` to return all contacts whose name **contains** a term (use `in`)
3. Add a `delete_contact(contacts, name)` function to `utils.py`
4. Prepare an `export_text(contacts)` function that formats all contacts as one

Thank You!

See you in Session 11 — File I/O and Persistent Apps!